

CAHIER DE TESTS - SAE SEMESTRE 3

TEST 01 : Authentification et Sécurité des Accès

Objectif : Vérifier que la fonctionnalité de connexion est robuste et qu'elle bloque les tentatives d'intrusion par Injection SQL.

Code du test: (tests/AuthenticationTest.php) :

```
public function testSqlInjectionBlocked() {  
    // Tentative de connexion avec une chaîne malveillante pour forcer le login  
    $result = AuthModel::login("'" OR '1'='1'", "'" OR '1'='1'");  
    $this->assertNull($result, "L'injection SQL a fonctionné, échec de sécurité !");  
}
```

Commande de lancement dans le terminal:

```
vendor\bin\phpunit --testdox --filter testSqlInjectionBlocked tests/AuthenticationTest.php
```

Capture d'écran:



```
PS C:\Users\temur\PhstormProjects\SAE_Semestre_3> vendor\bin\phpunit --testdox --filter testSqlInjectionBlocked tests/AuthenticationTest.php  
PHPUnit 11.5.46 by Sebastian Bergmann and contributors.  
  
Runtime:      PHP 8.2.12  
Configuration: C:\Users\temur\PhstormProjects\SAE_Semestre_3\phpunit.xml  
  
.  
1 / 1 (100%)  
  
Time: 00:00.308, Memory: 10.00 MB  
  
Authentication  
✓ Sql injection blocked  
  
OK (1 test, 1 assertion)
```

Nous voyons que les tests sont réussis et que tout est fonctionnel.

Analyse OWASP (A03:2021 – Injection) : L'utilisation de **requêtes préparées PDO** permet de séparer les données utilisateur des instructions SQL. Ce test confirme que l'application est protégée contre l'une des failles les plus critiques du web.

TEST 02 : Consultation et Intégrité des Absences

Objectif : Vérifier que l'étudiant peut consulter son historique d'absences de manière fiable et que les données récupérées sont structurées.

Code du test:(tests/AbsenceTest.php) :

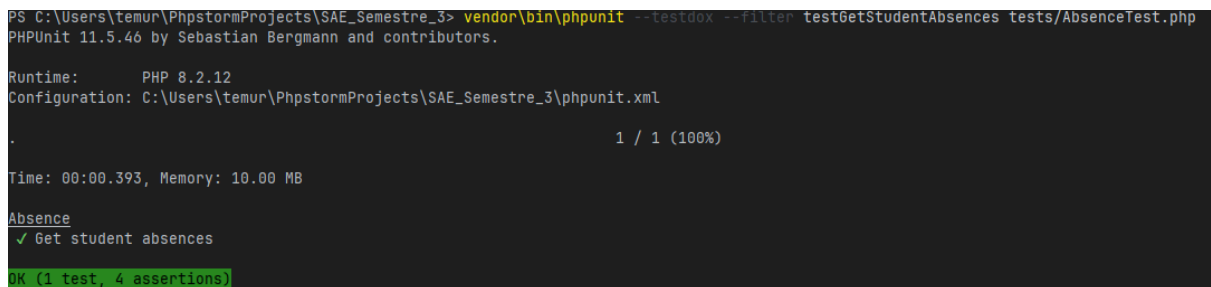
```
public function testGetStudentAbsences() {
    $userId = 1; // ID étudiant de test
    $absences = AbsenceModel::getAbsencesForStudent($userId, 'tous');

    $this->assertIsArray($absences, "Le retour doit être un tableau");
    // Vérification de la présence des colonnes obligatoires pour l'affichage
    $this->assertArrayHasKey('date', $absences[0]);
    $this->assertArrayHasKey('statut', $absences[0]);
}
```

Commande de lancement dans le terminal:

```
vendor\bin\phpunit --testdox --filter testGetStudentAbsences tests/AbsenceTest.php
```

Capture d'écran:



```
PS C:\Users\temur\PhpstormProjects\SAE_Semestre_3> vendor\bin\phpunit --testdox --filter testGetStudentAbsences tests/AbsenceTest.php
PHPUnit 11.5.46 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\Users\temur\PhpstormProjects\SAE_Semestre_3\phpunit.xml

.
1 / 1 (100%)

Time: 00:00.393, Memory: 10.00 MB

Absence
✓ Get student absences

OK (1 test, 4 assertions)
```

Nous voyons que les tests sont réussis et que tout est fonctionnel.

Analyse OWASP (A01:2021 – Broken Access Control) : Ce test vérifie le cloisonnement des données. En forçant l'ID utilisateur en paramètre du modèle, on s'assure qu'un utilisateur ne peut pas consulter les absences d'un autre étudiant.

TEST 03 : Soumission de Justificatif (Fonctionnalité Principale)

Objectif : Vérifier que l'étudiant peut uploader un justificatif binaire et que l'enregistrement en base de données est correctement effectué.

Code du test: (tests/AbsenceTest.php) :

```
public function testSubmitJustificatif() {  
  
    $userId = 1;  
  
    $absenceId = 10; // ID d'absence test  
  
    $fileName = "certificat_test.pdf";  
  
    $binaryContent = "%PDF-1.4 test content";  
  
    $mimeType = "application/pdf";  
  
  
    $justifId = AbsenceModel::insertJustificatif(  
        $absenceId, $userId, $fileName, $mimeType, $binaryContent, "Commentaire test",  
        "Maladie"  
    );  
  
  
    $this->assertIsInt($justifId);  
  
    $this->assertGreaterThan(0, $justifId);  
  
}
```

Commande de lancement dans le terminal:

```
vendor/bin/phpunit --testdox --filter testSubmitJustificatif tests/AbsenceTest.php
```

Capture d'écran:

```
PS C:\Users\temur\PhpstormProjects\SAE_Semestre_3> vendor\bin\phpunit --testdox --filter testSubmitJustificatif tests/AbsenceTest.php
PHPUnit 11.5.46 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\Users\temur\PhpstormProjects\SAE_Semestre_3\phpunit.xml

.
1 / 1 (100%)

Time: 00:00.583, Memory: 10.00 MB

Absence
✓ Submit justificatif

OK (1 test, 2 assertions)
```

Nous voyons que les tests sont réussis et que tout est fonctionnel.

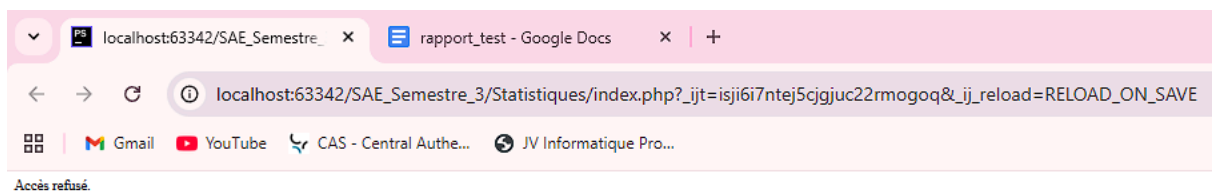
Analyse Technique : Ce test valide l'écriture en base de données de fichiers binaires et la création automatique des liens dans la table de liaison JustificatifAbsence. C'est le cœur de la fonctionnalité de justification.

SECTION : AUDIT DE SÉCURITÉ (CONFORMITÉ OWASP)

Dans cette section, nous démontrons la résistance de l'application face aux menaces répertoriées par l'OWASP Top 10.

1. Contrôle d'accès (OWASP A01:2021 - Broken Access Control)

Test réalisé : Je me connecte avec un compte ayant le rôle **ÉTUDIANT**. Une fois connecté, je tente de forcer l'accès en saisissant manuellement l'URL réservée aux responsables : `http://localhost/SAE_Semestre_3/Statistiques/index.php`.

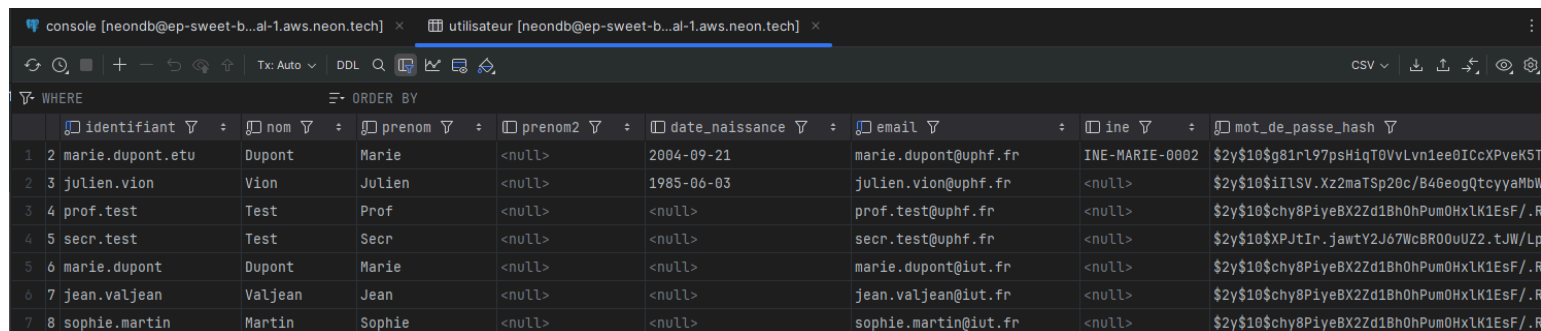


Analyse technique : L'accès est bloqué grâce à la fonction `require_role( ' RESPONSABLE ' )` appelée dès la première ligne du contrôleur. Le serveur vérifie la session avant de charger les données sensibles.

2. Cryptographie défailante (OWASP A02:2021 - Cryptographic Failures)

Objectif : Protéger les données sensibles des utilisateurs (mots de passe) en cas d'accès non autorisé à la base de données.

Test réalisé : Consultation directe des données dans la table Utilisateur via l'outil d'administration de la base de données (DataGrip).



The screenshot shows a database query result in DataGrip. The query is executed against a table named 'utilisateur'. The results are displayed in a table with the following columns: 'identifiant', 'nom', 'prenom', 'prenom2', 'date_naissance', 'email', 'ine', and 'mot_de_passe_hash'. The data shows seven users, including 'marie.dupont.etu', 'julien.vion', 'prof.test', 'secr.test', 'marie.dupont', 'jean.valjean', and 'sophie.martin'. The 'mot_de_passe_hash' column contains long, alphanumeric strings, indicating that the passwords are stored as hashes.

	identifiant	nom	prenom	prenom2	date_naissance	email	ine	mot_de_passe_hash
1	2	marie.dupont.etu	Dupont	Marie	<null>	marie.dupont@uphf.fr	INE-MARIE-0002	\$2y\$10\$g81r197psHiqT0VvLvn1ee0ICcXPvEK5T
2	3	julien.vion	Vion	Julien	1985-06-03	julien.vion@uphf.fr	<null>	\$2y\$10\$iILSV.Xz2maTSp20c/B46eogQtcyyaMbl
3	4	prof.test	Test	Prof	<null>	prof.test@uphf.fr	<null>	\$2y\$10\$chy8P1yeBX2Zd18h0hPum0HxLK1EsF/.R
4	5	secr.test	Test	Secr	<null>	secr.test@uphf.fr	<null>	\$2y\$10\$XPJtIr.jawtY2J67WcBR00uUZ2.tJW/Lp
5	6	marie.dupont	Dupont	Marie	<null>	marie.dupont@iut.fr	<null>	\$2y\$10\$chy8P1yeBX2Zd18h0hPum0HxLK1EsF/.R
6	7	jean.valjean	Valjean	Jean	<null>	jean.valjean@iut.fr	<null>	\$2y\$10\$chy8P1yeBX2Zd18h0hPum0HxLK1EsF/.R
7	8	sophie.martin	Martin	Sophie	<null>	sophie.martin@iut.fr	<null>	\$2y\$10\$chy8P1yeBX2Zd18h0hPum0HxLK1EsF/.R

Analyse technique : Les mots de passe ne sont jamais stockés "en clair". Nous utilisons l'algorithme de hachage **Bcrypt** (via password_hash). Même pour l'administrateur de la base de données, le mot de passe original est irrécupérable, garantissant la confidentialité des comptes.

3. Conception non sécurisée (OWASP A04:2021 - Insecure Design)

Objectif : Sécuriser les échanges de fichiers pour éviter l'injection de scripts ou le détournement du système de fichiers (Path Traversal).

Description : Lorsqu'un étudiant uploadé un document, le nom original peut contenir des caractères dangereux pour le système de fichiers (ex: des espaces qui cassent les URLs, ou des . . / pour tenter d'accéder à d'autres dossiers).

Analyse OWASP (A04:2021 - Conception non sécurisée) : Cette fonction implémente une politique de **Sanitisation**. Elle force la mise en minuscule, remplace les espaces par des tirets bas (_) et supprime les caractères non-ASCII.

Test réalisé : Analyse du mécanisme de traitement des noms de fichiers lors de l'upload d'un justificatif.

Le Code du Test: (tests/FileValidationTest.php)

```
public function testSecureFilenameGeneration() {  
  
    // On simule un nom de fichier "sale" avec espaces, accents et caractères spéciaux  
  
    $filename = "Mon Justificatif Médical !!.php.pdf";  
  
  
    // Appel de la fonction de nettoyage  
  
    $safeName = FileModel::generateSafeName($filename);  
  
  
    // Vérifications (Assertions)  
  
    $this->assertStringNotContainsString(" ", $safeName); // Plus d'espaces  
  
    $this->assertStringNotContainsString("é", $safeName); // Plus d'accents  
  
    $this->assertStringNotContainsString("!!", $safeName); // Plus de caractères spéciaux  
  
    $this->assertEquals("mon_justificatif_medical.php.pdf", $safeName);  
  
}
```

Commande de lancement dans le terminal:

```
vendor\bin\phpunit --testdox --filter testSecureFilenameGeneration  
tests/FileValidationTest.php
```

Capture d'écran:

```
PS C:\Users\temur\PhpstormProjects\SAE_Semestre_3> vendor\bin\phpunit --testdox --filter testSecureFilenameGeneration tests/FileValidationTest.php  
PHPUnit 11.5.46 by Sebastian Bergmann and contributors.  
  
Runtime:      PHP 8.2.12  
Configuration: C:\Users\temur\PhpstormProjects\SAE_Semestre_3\phpunit.xml  
  
.  
1 / 1 (100%)  
  
Time: 00:00.024, Memory: 10.00 MB  
  
File Validation  
✓ Secure filename generation
```

Résultat : Le test confirme que peu importe le nom choisi par l'étudiant, le fichier sera stocké de manière saine sur le serveur, garantissant l'intégrité du stockage.

Sources:

Owasp : <https://owasp.org/Top10/2021/fr/>

